PLANO DE DIVISÃO DE TRABALHO

Implementação do Jogo da Forca em Flutter e Dart

1ª Avaliação - Programação para Dispositivos Móveis

Curso de Tecnologia em Sistemas de Computação - UESPI

Professor: Eyder Rios

Objetivo do documento

Distribuir as responsabilidades entre quatro integrantes de forma equilibrada, reduzir conflitos de implementação e garantir cobertura explícita dos requisitos funcionais, regras de negócio, critérios técnicos e itens de avaliação do enunciado.

Documento interno da equipe - versão de planejamento

1. Princípios usados na divisão

A divisão foi estruturada a partir do enunciado da avaliação e do arquivo JSON fornecido pelo professor. O JSON é tratado como fonte oficial de dados e não como uma feature a ser criada pela equipe.

> **Regra central sobre os dados**
>
> A equipe deve incorporar o arquivo JSON aos assets, carregá-lo em tempo de execução, desserializá-lo e converter seus campos para objetos Dart. Não deve copiar manualmente as respostas para listas, mapas ou constantes do código-fonte.

- Cada integrante possui uma área principal de autoria, com arquivos e responsabilidades bem definidos.
- As áreas foram separadas por responsabilidade técnica: dados, domínio, apresentação e sessão.
- Nenhum integrante deve duplicar regras que já pertencem a outro módulo.
- Mudanças em contratos compartilhados devem ser alinhadas antes de serem implementadas.
- Todos devem possuir commits efetivos no repositório e conhecimento geral de toda a solução.
- A divisão interna não elimina a responsabilidade coletiva na apresentação e na arguição.

2. Visão geral da distribuição

Integrante	Área principal	Branch sugerida	Carga técnica
Integrante 1	Dados, modelagem e configuração da partida	feature/data-and-setup	Alta
Integrante 2	Motor do jogo e regras de negócio	feature/game-domain	Alta
Integrante 3	Interface da partida e responsividade	feature/game-presentation	Alta
Integrante 4	Sessão, sorteio, resultados e estatísticas	feature/session-and-results	Alta

A distribuição evita separar o projeto apenas por telas. O objetivo é preservar coesão: cada integrante concentra um conjunto de responsabilidades que fazem sentido juntas e expõe contratos claros para os demais.

2.1 Equilíbrio de dificuldade

Integrante	Principais fontes de complexidade	Nível relativo
1	JSON, enums obrigatórios, modelagem, repository, filtros e configuração da partida	4/5
2	Regras centrais, normalização, estados, vitória/derrota e testes unitários	5/5
3	Tela principal, teclado, forca progressiva, feedback e responsividade	4,5/5
4	Sorteio sem repetição, ciclo da sessão, estatísticas, resultado e navegação	4,5/5

O Integrante 2 possui a maior densidade lógica. Para equilibrar, ele não deve assumir navegação, telas, carregamento do JSON ou estatísticas da sessão. O Integrante 1 assume também a documentação técnica relacionada aos dados e à configuração. O Integrante 3 concentra a maior carga visual. O Integrante 4 concentra o ciclo entre rodadas e o estado acumulado da sessão.

3. Integrante 1 - Dados, modelagem e configuração

Missão principal: transformar o JSON oficial em objetos de domínio válidos e fornecer à aplicação uma configuração de partida consistente, sem assumir regras do jogo ou estatísticas.

3.1 Responsabilidades

Atividade	Entrega esperada
Enums obrigatórios	Criar AnswerCategory e Difficulty com os valores técnicos definidos no enunciado.
Modelo AnswerEntry	Representar answer, category, difficulty e hints preservando a resposta original.
Desserialização	Converter o JSON para objetos Dart e transformar category/difficulty em enums.
Asset local	Registrar o arquivo JSON no pubspec.yaml e carregá-lo com recurso local.
Tratamento de erro	Tratar falha de leitura, estrutura inválida ou valor enumerado inesperado.
Repository	Disponibilizar consultas de respostas por categoria e dificuldade.
Textos de interface	Converter valores técnicos para português somente na apresentação.
Configuração da partida	Implementar seleção de categoria, dificuldade e confirmação antes de iniciar.
Documentação técnica	Documentar no README como os dados são carregados e como executar o projeto.

3.2 Requisitos sob responsabilidade principal

- RF01 - Início do jogo.
- RF02 - Categorias e níveis de dificuldade.
- Seção 3 - Dados fornecidos em JSON.
- Seção 3.1 - Valores enumerados obrigatórios.
- Seção 3.2 - Carregamento e desserialização.
- Parte de RNF04 relacionada a nomenclatura técnica e textos apresentados ao usuário.

3.3 Casos do JSON que devem ser usados na validação

Entrada real	O que validar
MÃO DIREITA	Acento e espaço são preservados.
CLIENTE-SERVIDOR	Hífen permanece parte da resposta original.
ARTICULAÇÃO TÊMPORO-MANDIBULAR	Resposta longa, acentos e hífen chegam intactos ao domínio.

Entrada real	O que validar
CORRIDA COM OBSTÁCULOS	Expressão com várias palavras é carregada sem fragmentação indevida.

3.4 Limites de responsabilidade

- Não implementar lógica de acerto, erro, vitória ou derrota.
- Não controlar estatísticas da sessão.
- Não implementar o desenho progressivo da forca.
- Não manter uma cópia paralela das respostas dentro do código.
- Não alterar os valores técnicos de category e difficulty definidos no JSON.

3.5 Critério de conclusão

- O aplicativo carrega o JSON localmente sem Internet.
- Todas as categorias e dificuldades válidas viram enums no domínio.
- Os filtros retornam somente respostas compatíveis com a seleção.
- A tela de configuração permite escolher categoria e dificuldade e deixa a seleção clara.
- flutter analyze não aponta erros ou advertências introduzidos pela feature.

4. Integrante 2 - Motor do jogo e regras de negócio

Missão principal: implementar as regras centrais do Jogo da Forca de forma independente da interface. Esta feature deve poder ser testada sem depender de widgets Flutter.

4.1 Responsabilidades

Atividade	Entrega esperada
Estado da rodada	Representar rodada em andamento, vitória e derrota com enum apropriado.
Tentativas	Registrar letras utilizadas e impedir efeito de tentativa repetida.
Acertos	Revelar todas as ocorrências da letra na resposta completa.
Erros	Incrementar exatamente um erro por tentativa incorreta.
Normalização	Comparar letras ignorando acentuação sem modificar a grafia original.
Caracteres estruturais	Tratar espaços, hífens e pontuação como visíveis e não jogáveis.
Condição de vitória	Detectar quando todas as letras foram identificadas antes do limite.
Condição de derrota	Encerrar a rodada ao atingir o limite máximo de erros.
Bloqueio pós-encerramento	Não aceitar tentativa depois de vitória ou derrota.
Dica na rodada	Controlar se a dica já foi utilizada para impedir contabilização duplicada.
Resultado	Produzir RoundResult com informações necessárias para a tela e para a sessão.
Testes unitários	Cobrir regras centrais e casos de contorno.

4.2 Requisitos sob responsabilidade principal

- RF05 - Tentativa de letras.
- RF06 - Acentuação e caracteres não alfabéticos.
- RF07 - Controle de erros.
- RF09 - Parte lógica do sistema de dicas.
- RF10 - Condição de vitória.
- RF11 - Condição de derrota.
- Seção 4 - Regras de negócio obrigatórias.

> **Regra crítica**
> A normalização deve existir somente para comparação. Se a resposta contiver Ã, a tentativa A deve revelar essa posição, mas a interface deve continuar mostrando Ã. A resposta original nunca deve ser substituída por uma versão sem acentos.

4.3 Testes unitários mínimos

Cenário	Resultado esperado
Letra correta	Revela todas as ocorrências.
Letra incorreta	Incrementa um único erro.
Letra repetida	Não altera erros nem estado.
A contra Ã	Deve ser considerada correspondência.
Espaço ou hífen	Já aparece e não constitui tentativa.
Vitória	Ocorre apenas quando todas as letras necessárias foram reveladas.
Limite máximo	Produz derrota e encerra a rodada.
Tentativa após encerramento	É ignorada.
Dica consultada novamente	Não gera segunda contabilização.

4.4 Limites de responsabilidade

- Não desenhar widgets da tela principal.
- Não controlar respostas já usadas na sessão.
- Não carregar o arquivo JSON diretamente.
- Não calcular percentuais acumulados de vitórias e derrotas.
- Não assumir navegação entre telas.

5. Integrante 3 - Interface da partida e responsividade

Missão principal: apresentar o estado da rodada de forma clara e responsiva. A camada visual consome o estado fornecido pelo domínio; ela não deve recriar as regras do jogo.

5.1 Responsabilidades

Componente/atividade	Entrega esperada
GameScreen	Compor a tela principal da partida.
AnswerBoard	Exibir a resposta parcialmente revelada e preservar espaços/hífens.

Componente/atividade	Entrega esperada
LetterKeyboard	Disponibilizar somente letras e distinguir disponíveis de utilizadas.
HangmanFigure	Mostrar progressão visual da forca conforme o número de erros.
ErrorProgress	Exibir quantidade ou progresso dos erros.
HintButton/HintView	Permitir solicitar e visualizar a dica atual.
Informações da rodada	Exibir categoria e dificuldade atuais.
Feedback visual	Atualizar a interface após cada tentativa.
Responsividade	Evitar overflow, sobreposição, corte de texto e controles inacessíveis.
Qualidade visual	Manter hierarquia, legibilidade e consistência entre componentes.

5.2 Requisitos sob responsabilidade principal

- RF08 - Representação gráfica progressiva da forca.
- Parte visual de RF05, RF06, RF07 e RF09.
- Seção 8.2 - Tela principal do jogo.
- RNF01 - Usabilidade.
- RNF02 - Interface responsiva.
- RNF03 - Qualidade visual.

5.3 Estados mínimos da forca

1. Cabeça.
2. Tronco.
3. Braço esquerdo.
4. Braço direito.
5. Perna esquerda.
6. Perna direita.

A quantidade máxima de erros deve ser compatível com os estados gráficos implementados. Cada erro precisa produzir uma alteração visual perceptível.

5.4 Casos reais para testar responsividade

Resposta do JSON	Objetivo do teste
ARTICULAÇÃO TÊMPORO-MANDIBULAR	Maior pressão de largura, hífen e acentuação.
CORRIDA COM OBSTÁCULOS	Quebra entre várias palavras.
CONTROLE DE VERSÃO	Expressão composta em largura intermediária.
QUATI SUL-AMERICANO	Combinação de espaço e hífen.

5.5 Limites de responsabilidade

- Não decidir se uma tentativa é correta ou incorreta dentro do widget.
- Não normalizar acentos na camada visual.
- Não calcular vitória ou derrota no GameScreen.

- Não controlar estatísticas acumuladas.
- Não acessar diretamente o JSON para escolher respostas.

6. Integrante 4 - Sessão, sorteio, resultados e estatísticas

Missão principal: controlar o ciclo entre várias rodadas enquanto a aplicação permanece em execução. A sessão não é uma conta de usuário; é o estado acumulado enquanto o aplicativo está aberto.

6.1 Responsabilidades

Atividade	Entrega esperada
Sorteio	Selecionar aleatoriamente uma resposta compatível com categoria e dificuldade.
Não repetição	Evitar resposta já usada enquanto houver outra disponível na mesma combinação.
Esgotamento	Informar quando todas as respostas daquela combinação já foram utilizadas.
SessionManager	Manter respostas usadas e estatísticas durante a execução da aplicação.
SessionStatistics	Acumular rodadas, vitórias, derrotas, erros e dicas.
Percentuais	Calcular vitórias e derrotas a partir de rodadas concluídas.
ResultScreen	Apresentar vitória/derrota, resposta, erros e uso de dica.
Nova rodada	Permitir jogar novamente sem perder a sessão.
Retorno à configuração	Permitir nova seleção de categoria e dificuldade.
StatisticsScreen	Disponibilizar consulta clara das estatísticas acumuladas.
AboutScreen	Exibir os nomes completos dos integrantes.
Navegação	Integrar configuração, jogo, resultado, estatísticas e sobre.

6.2 Requisitos sob responsabilidade principal

- RF03 - Sorteio da resposta.
- RF04 - Não repetição de respostas.
- RF12 - Nova rodada.
- RF13 - Estatísticas da sessão.
- RF14 - Identificação dos membros da equipe.
- Seções 8.3, 8.4 e 8.5 - Resultado, estatísticas e identificação.

6.3 Estrutura conceitual do controle de respostas usadas

```
humanBody + easy     -> { respostas já usadas }
technology + hard    -> { respostas já usadas }
animals + medium     -> { respostas já usadas }
sports + easy        -> { respostas já usadas }
```

A implementação pode usar Map, Set ou estrutura equivalente. O importante é que o controle considere a combinação de categoria e dificuldade, e não apenas a resposta isoladamente.

6.4 Estatísticas mínimas

Indicador	Regra
Rodadas realizadas	Somente rodadas concluídas.
Vitórias	Quantidade e percentual.
Derrotas	Quantidade e percentual.
Erros	Total acumulado nas rodadas concluídas.
Dicas utilizadas	Contabilizar no máximo uma vez por rodada.

6.5 Limites de responsabilidade

- Não reimplementar a validação de letras.
- Não decidir vitória ou derrota fora do domínio da rodada.
- Não desserializar o JSON.
- Não desenhar a forca.
- Não persistir as estatísticas em banco, arquivo remoto ou serviço externo.

7. Contratos entre os integrantes

Os contratos reduzem dependências informais. Antes de integrar as branches, a equipe deve concordar com os tipos públicos e com o sentido de cada campo.

7.1 Integrante 1 entrega para o projeto

```
AnswerEntry
AnswerCategory
Difficulty
AnswerRepository
```

Esses tipos permitem que os demais módulos trabalhem com dados válidos sem conhecer detalhes do arquivo JSON.

7.2 Integrante 2 entrega para apresentação e sessão

```
GameRound
RoundStatus
RoundResult
```

RoundResult deve expor somente o que as outras features realmente precisam. Exemplo conceitual: resultado da rodada, quantidade de erros e indicação de uso de dica.

7.3 Integrante 3 consome estado; não calcula regra

Recebe	Apresenta
Resposta/estado visível	Letras descobertas e caracteres estruturais.
Letras utilizadas	Estado visual do teclado.
Número de erros	Forca e progresso.

Recebe	Apresenta
Status da rodada	Feedback visual de estado.
Dica	Conteúdo solicitado pelo jogador.

7.4 Integrante 4 consome RoundResult

O módulo de sessão usa o resultado encerrado para atualizar as estatísticas. Ele não deve analisar novamente a palavra para decidir se houve vitória ou derrota.

> **Regra para alterações de contrato**
> Se uma feature precisar de um dado que o contrato atual não fornece, o integrante responsável pelo contrato deve realizar a alteração. Evitem editar diretamente arquivos que pertencem a outro módulo apenas para resolver uma necessidade local.

8. Propriedade principal de arquivos e prevenção de conflitos

Área/arquivo	Responsável principal
pubspec.yaml e registro do JSON	Integrante 1
data/models, data/repositories e data/sources	Integrante 1
features/setup	Integrante 1
features/game/domain	Integrante 2
Normalização de letras e regras da rodada	Integrante 2
features/game/presentation	Integrante 3
Widgets da forca e teclado	Integrante 3
Tema visual relacionado à partida	Integrante 3
features/session	Integrante 4
features/result	Integrante 4
features/statistics	Integrante 4
features/about	Integrante 4
app.dart e navegação final	Integrante 4, com revisão coletiva
README.md final	Integrante 1, com revisão dos quatro

9. Estrutura sugerida do projeto

A estrutura abaixo não é uma arquitetura obrigatória. Ela apenas materializa a separação pedida no enunciado entre dados, regras, estado e apresentação, sem criar camadas artificiais.

```
lib/
  main.dart
  app.dart
  data/  -> models, repositories, sources
```

```
domain/  -> enums e utilitários compartilhados
features/
  setup/
  game/domain/
  game/presentation/
  session/
  result/
  statistics/
  about/
shared/
```

10. Ordem recomendada de desenvolvimento e integração

1. Definir os contratos mínimos: AnswerEntry, enums, GameRound, RoundStatus e RoundResult.
2. Integrante 1 implementa carregamento/modelagem enquanto os demais trabalham usando contratos acordados.
3. Integrante 2 implementa regras de negócio e testes unitários sem depender da tela.
4. Integrante 3 implementa os widgets da partida usando estados simulados quando necessário.
5. Integrante 4 implementa sessão, estatísticas, telas de resultado e estrutura de navegação.
6. Integrar primeiro data-and-setup.
7. Integrar game-domain e executar os testes das regras.
8. Integrar session-and-results.
9. Integrar game-presentation.
10. Executar testes manuais com respostas longas, acentuadas, com espaços e com hífens.
11. Executar dart format . e flutter analyze.
12. Revisar README, histórico de commits e funcionamento offline antes da entrega.

10.1 Política simples para Git

- Uma branch principal por feature, com commits pequenos e descritivos.
- Evitar commits que misturem responsabilidades de duas features.
- Antes do merge, atualizar a branch com a main e resolver conflitos na própria branch.
- Pull request deve indicar quais requisitos foram atendidos e como foram testados.
- Mudanças em arquivos compartilhados devem ser pequenas e justificadas.
- Todos os integrantes devem possuir contribuições reais e identificáveis no histórico.

11. Matriz de cobertura dos requisitos funcionais

Requisito	Responsável principal	Apoio/integração
RF01 - Início do jogo	Integrante 1	Integrante 4 na navegação
RF02 - Categorias e dificuldade	Integrante 1	Integrante 4 na navegação
RF03 - Sorteio da resposta	Integrante 4	Integrante 1 fornece as respostas filtradas
RF04 - Não repetição	Integrante 4	Integrante 1 fornece dados; Integrante 2 recebe a resposta
RF05 - Tentativa de letras	Integrante 2	Integrante 3 apresenta teclado e feedback
RF06 - Acentuação/caracteres	Integrante 2	Integrante 3 preserva a representação visual

Requisito	Responsável principal	Apoio/integração
RF07 - Controle de erros	Integrante 2	Integrante 3 apresenta progresso/forca
RF08 - Forca progressiva	Integrante 3	Integrante 2 fornece contagem de erros
RF09 - Dicas	Integrante 2 + 3	Integrante 4 contabiliza na sessão
RF10 - Vitória	Integrante 2	Integrante 4 recebe resultado; Integrante 3 apresenta estado
RF11 - Derrota	Integrante 2	Integrante 4 recebe resultado; Integrante 3 apresenta estado
RF12 - Nova rodada	Integrante 4	Integrantes 1 e 2 fornecem configuração e rodada
RF13 - Estatísticas	Integrante 4	Integrante 2 fornece RoundResult
RF14 - Identificação da equipe	Integrante 4	Revisão coletiva dos nomes

12. Definition of Done por feature

Uma feature só deve ser considerada pronta quando atender aos quatro pontos abaixo:

- Funcionalidade: comportamento implementado conforme o requisito correspondente.
- Qualidade: responsabilidades separadas, nomes técnicos em inglês e ausência de duplicação desnecessária.
- Validação: testes unitários ou roteiro de teste manual adequado à feature.
- Integração: dart format . aplicado e flutter analyze sem problemas provenientes da alteração.

13. Checklist final antes da entrega

Item	Verificação
JSON	Está nos assets, é carregado em tempo de execução e não foi copiado manualmente para o código.
Enums	Category, difficulty e demais conjuntos fechados relevantes usam enum.
Respostas compostas	Espaços, hífens e pontuação aparecem automaticamente.
Acentuação	Comparação ignora acento, mas exibição preserva a grafia original.
Tentativas	Somente letras podem ser escolhidas e repetição não gera novo efeito.
Forca	Cada erro altera o estado gráfico e o limite é compatível com os estados.
Não repetição	Resposta usada não é sorteada novamente enquanto houver opção disponível.
Sessão	Estatísticas e respostas usadas permanecem entre rodadas enquanto o app executa.
Estatísticas	Totais e percentuais estão corretos e dica conta no máximo uma vez por rodada.
Responsividade	Sem overflow, corte, sobreposição ou controle inacessível.

Item	Verificação
Offline	Jogo funciona sem Internet, backend, banco ou API.
Código	dart format . executado e flutter analyze sem erros ou advertências do código da equipe.
README	Começa com os nomes completos e contém descrição, funcionalidades, tecnologias e execução.
GitHub	Repositório público, main como principal, commits efetivos de todos e .gitignore adequado.

14. Preparação para a apresentação e arguição

A autoria principal serve para organizar o desenvolvimento, não para limitar o conhecimento. O professor poderá solicitar que qualquer integrante explique ou modifique partes desenvolvidas por outra pessoa.

14.1 Conteúdo que todos devem saber explicar

- Como o JSON é carregado e desserializado.
- Por que category e difficulty são enums.
- Como respostas com várias palavras são representadas.
- Como espaços, hífens e outros caracteres são tratados.
- Como a comparação ignora acentuação sem alterar a resposta original.
- Como uma tentativa correta revela todas as ocorrências.
- Como vitória e derrota são determinadas.
- Como a aplicação evita repetição de respostas na mesma sessão.
- Como as estatísticas são atualizadas e como os percentuais são calculados.
- Como a arquitetura separa dados, regras, estado e interface.
- Onde localizar cada responsabilidade no código-fonte.

14.2 Revisão cruzada recomendada

Quem revisa	Feature principal a revisar
Integrante 1	Game domain do Integrante 2
Integrante 2	Game presentation do Integrante 3
Integrante 3	Session/results do Integrante 4
Integrante 4	Data/setup do Integrante 1

Depois da revisão cruzada, cada integrante deve conseguir descrever o fluxo completo: configuração -> sorteio -> rodada -> resultado -> atualização da sessão -> nova rodada.

15. Base documental utilizada

Este plano foi elaborado exclusivamente a partir de:

- Enunciado “1ª Avaliação - Trabalho de Implementação - Jogo da Forca”, da disciplina Programação para Dispositivos Móveis.
- Arquivo JSON oficial fornecido com a atividade, contendo as entradas de respostas, categorias, dificuldades e dicas.

Decisão de projeto: a organização interna sugerida é uma proposta de divisão de responsabilidades. Ela respeita os requisitos do enunciado e não impõe uma arquitetura rígida além do que a avaliação exige.